📘 BAB 3 — MEMBUAT PROJECT DENGAN REACT DAN INERTIA

Pada bab ini kita mulai membuat aplikasi Laravel bernama **Prezence**.

Frontend aplikasi akan dibuat menggunakan React dan MUI. Laravel tetap digunakan untuk route, controller, authentication, dan database. Inertia digunakan sebagai penghubung antara Laravel dan React.

Pada bab ini, Blade tidak digunakan untuk membuat halaman frontend. Blade akan digunakan pada bab lain jika kita perlu membuat template email Laravel.

3.1 Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

- membuat project Laravel baru menggunakan Laravel Herd;
- menyiapkan React dan Inertia pada project Laravel;
- memasang MUI v9 dan Day.js;
- mengenali struktur frontend React;
- membuat `MainLayout` menggunakan MUI;
- membuat halaman `About` dan `Guide`;
- memahami hubungan Laravel route dengan Inertia page;
- menjalankan migration dasar authentication;
- membuat form login menggunakan React dan MUI;
- memvalidasi email dan password melalui Laravel controller;
- menggunakan middleware `auth` untuk melindungi route;
- menampilkan dashboard setelah login.

3.2 Membuat Project Laravel

Pastikan Laravel Herd sudah terpasang dan perintah `laravel` dapat digunakan pada PowerShell.

Masuk ke folder tempat project Laravel disimpan:

```
cd ~/Herd
```

Buat project baru:

```
laravel new prezence
```

Laravel Installer akan menampilkan beberapa pertanyaan. Gunakan pilihan berikut:

Pertanyaan 1 — Starter Kit

```
Do you want to use a starter kit?
```

Pilih:

```
No
```

Pada tutorial ini, authentication akan dibuat secara bertahap agar siswa memahami proses login, controller, session, dan middleware.

Pertanyaan 2 — Frontend Stack

```
Which frontend stack do you want to build on?
```

Pilih:

```
React
```

Pilihan ini membuat project menggunakan React, Inertia, dan TypeScript. Karena itu, file frontend yang dihasilkan menggunakan ekstensi `.tsx`, bukan `.jsx`.

Pertanyaan Berikutnya

Gunakan pilihan berikut jika ditampilkan:

- menggunakan database SQLite;
- gunakan pilihan testing default yang tersedia;
- gunakan konfigurasi frontend default lainnya.

Setelah proses instalasi selesai, periksa file yang dibuat:

```
resources/js/app.tsx
resources/js/pages/welcome.tsx
tsconfig.json
vite.config.ts
```

Jika file-file tersebut tersedia, berarti project menggunakan React + TypeScript.

Masuk ke folder project:

```
cd prezence
```

Jika project berada di dalam folder Herd, buka alamat berikut pada browser:

```
http://prezence.test
```

Nama folder project menggunakan huruf kecil `prezence`, sedangkan nama aplikasi yang ditampilkan kepada pengguna adalah `Prezence`.

3.3 Mengenali Struktur Project

Beberapa folder penting pada project Laravel adalah:

Folder	Fungsi
`app`	Menyimpan Model, Controller, dan komponen aplikasi
`database`	Menyimpan migration, seeder, dan factory
`public`	Titik akses aplikasi dan asset publik
`resources/js`	Menyimpan kode React
`resources/views`	Digunakan oleh Blade, misalnya untuk email
`routes`	Menyimpan definisi route Laravel
`.env`	Menyimpan konfigurasi lingkungan lokal

Frontend utama pada bab ini berada di:

```
resources/js
```

3.4 Memeriksa Inertia React

Inertia menghubungkan Laravel dengan React. Laravel tetap menentukan route dan controller, sedangkan React menampilkan page pada browser.

Karena pada langkah sebelumnya kita memilih frontend stack `React`, Laravel Installer biasanya sudah memasang adapter Inertia, React, dan TypeScript. Periksa package yang tersedia pada `package.json` dan `composer.json` sebelum memasang package secara manual.

Periksa package Laravel:

```
composer show inertiajs/inertia-laravel
```

Periksa package frontend:

```
npm list @inertiajs/react react react-dom typescript
```

Jika package tersebut sudah tersedia, tidak perlu menjalankan `composer require` atau `npm install` ulang.

Package Inertia Laravel terpasang melalui Composer. Pada beberapa setup Laravel atau starter kit, middleware Inertia mungkin sudah dibuat dan didaftarkan secara otomatis. Karena itu, periksa terlebih dahulu sebelum membuat atau mendaftarkan middleware secara manual.

Periksa apakah command Inertia tersedia:

```
php artisan list | Select-String inertia
```

Periksa apakah file middleware sudah tersedia:

```
Test-Path app/Http/Middleware/HandleInertiaRequests.php
```

Jika hasil pemeriksaan menunjukkan file tersebut belum tersedia, buat middleware dengan perintah:

```
php artisan inertia:middleware
```

Memeriksa Pendaftaran Middleware Inertia

Periksa apakah `HandleInertiaRequests` sudah terdaftar pada `bootstrap/app.php`:

```
Select-String `
    -Path bootstrap/app.php `
    -Pattern "HandleInertiaRequests"
```

Jika belum terdaftar, buka file `bootstrap/app.php` dan tambahkan middleware tersebut pada middleware web:

```
use App\Http\Middleware\HandleInertiaRequests;
use Illuminate\Foundation\Configuration\Middleware;
```

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->web(append: [
        HandleInertiaRequests::class,
    ]);
})
```

Middleware ini membantu Laravel menyediakan data request kepada halaman Inertia.

Jika file dan pendaftaran middleware sudah tersedia, langkah pembuatan dan pendaftaran manual tidak perlu dilakukan lagi. Pastikan hanya ada satu pendaftaran `HandleInertiaRequests` agar middleware tidak tercatat dua kali.

Root View Inertia

Inertia tetap membutuhkan satu file Blade sebagai root view untuk memasang aplikasi React. File ini hanya berfungsi sebagai shell HTML dan bukan tempat membuat halaman frontend.

Periksa apakah file berikut sudah dibuat oleh installer:

```
resources/views/app.blade.php
```

Jika file belum tersedia, buat file tersebut. Isi atau sesuaikan dengan:

```
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}"
@class(['dark' => ($appearance ?? 'system') == 'dark'])>
    <head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-
scale=1">

        <link rel="icon" href="/favicon.ico" sizes="any">
        <link rel="icon" href="/favicon.svg" type="image/svg+xml">
        <link rel="apple-touch-icon" href="/apple-touch-icon.png">

        @fonts

        @viteReactRefresh
        @vite(['resources/css/app.css', 'resources/js/app.tsx',
"resources/js/pages/{$page['component']}.tsx"])
        <x-inertia::head>
            <title>{{ config('app.name', 'Laravel') }}</title>
        </x-inertia::head>
    </head>
    <body class="font-sans antialiased">
        <x-inertia::app />
    </body>
```

```
</html>
```

File Blade ini hanya digunakan sebagai titik pemasangan React. Halaman `About`, `Guide`, `Login`, dan `Dashboard` tetap dibuat sebagai React page.

3.5 Memasang MUI v9 dan Day.js

MUI menyediakan komponen frontend siap pakai untuk React. MUI menggunakan Emotion sebagai styling engine.

Pasang MUI dan dependency-nya:

```
npm install @mui/material@^9 @emotion/react @emotion/styled
```

Pasang Day.js:

```
npm install dayjs
```

Day.js akan digunakan untuk mengolah tanggal, misalnya menampilkan tanggal hari ini pada form Attendance.

Jika nanti membutuhkan date picker dari MUI, pasang package tambahan:

```
npm install @mui/x-date-pickers
```

> MUI adalah library React. Karena itu, MUI tidak digunakan langsung pada file Blade. Blade hanya digunakan untuk kebutuhan tertentu seperti template email.

3.6 Memeriksa Konfigurasi Vite dan TypeScript

React, TypeScript, plugin React, dan Vite sudah disiapkan oleh Laravel Installer ketika frontend stack `React` dipilih. Karena itu, kita tidak perlu memasang ulang plugin React atau mengganti konfigurasi Vite secara manual.

Periksa file konfigurasi berikut:

```
vite.config.ts
tsconfig.json
```

File entry point React pada project ini adalah:

```
resources/js/app.tsx
```

Laravel Installer sudah membuat konfigurasi entry point Inertia pada file tersebut. Jangan mengganti seluruh isi `app.tsx` secara manual. Pertahankan konfigurasi yang dihasilkan installer agar tetap kompatibel dengan Inertia, Vite Plus, dan plugin Wayfinder.

Pada project ini, page React disimpan pada folder:

```
resources/js/pages
```

Menambahkan Alias Import `@`

Project Laravel sudah memiliki konfigurasi alias pada `tsconfig.json`:

```
{
    "compilerOptions": {
        "paths": {
            "@/*": ["./resources/js/*"]
        }
    }
}
```

Konfigurasi tersebut membuat TypeScript mengenali alias `@`. Agar Vite juga dapat mengenalinya saat proses development dan build, pasang plugin `vite-tsconfig-paths`:

```
npm install -D vite-tsconfig-paths
```

Buka `vite.config.ts`, lalu tambahkan import berikut:

```
import tsconfigPaths from 'vite-tsconfig-paths';
```

Tambahkan `tsconfigPaths()` pada daftar plugin. Jangan menghapus plugin lain yang sudah dibuat oleh Laravel Installer:

```
export default defineConfig({
    plugins: lazyPlugins(() => [
        tsconfigPaths(),

        laravel({
            input: ['resources/css/app.css', 'resources/js/app.tsx'],
            refresh: true,
```

```
        }),

        inertia(),
        react(),
        tailwindcss(),
        wayfinder(),
    ]),
});
```

Setelah itu, import dapat menggunakan alias:

```
import MainLayout from '@/layouts/MainLayout';
```

Alias tersebut menunjuk ke:

```
@/layouts/MainLayout
        ↓
resources/js/layouts/MainLayout.tsx
```

Periksa konfigurasi dengan perintah:

```
npm run types:check
npm run build
```

Jalankan Vite pada terminal:

```
npm run dev
```

Biarkan proses ini berjalan selama mengembangkan frontend.

3.7 Membagikan Data User ke React

`MainLayout` perlu mengetahui apakah User sedang login agar dapat menampilkan button `Logout` secara kondisional.

Buka file:

```
app/Http/Middleware/HandleInertiaRequests.php
```

Pastikan method `share()` membagikan data User:

```
public function share(Request $request): array
{
    return [
        ...parent::share($request),
        'auth' => [
            'user' => $request->user(),
        ],
    ];
}
```

Jika User belum login, nilai `auth.user` adalah `null`. Jika User sudah login, data User tersedia pada setiap React page melalui Inertia.

3.8 Membuat MainLayout dengan MUI

Buat folder:

```
resources/js/layouts
```

Buat file:

```
resources/js/layouts/MainLayout.tsx
```

Isi dengan layout sederhana:

```
import { AppBar, Button, Container, Toolbar, Typography } from
"@mui/material";
import {  usePage, router } from "@inertiajs/react";
import type { ReactNode } from "react";

export default function MainLayout({ children }: { children: ReactNode
}) {
    const { auth } = usePage().props;
    const user = auth?.user;

    return (
        <>
            <AppBar position="static">
                <Toolbar>
                    <Typography variant="h6" sx={{ flexGrow: 1 }}>
                        Prezence
                    </Typography>
                    {user && (
                        <Button
                            color="inherit"
```

```
                            onClick={() => router.visit("/")}
                        >
                            Dashboard
                        </Button>
                    )}

                    <Button
                        color="inherit"
                        onClick={() => router.visit("/about")}
                    >
                        About
                    </Button>
                    <Button
                        color="inherit"
                        onClick={() => router.visit("/guide")}
                    >
                        Guide
                    </Button>
                    {user ? (
                        <Button
                            color="inherit"
                            onClick={() => router.post("/logout")}
                        >
                            Logout
                        </Button>
                    ):(
                        <Button
                            color="inherit"
                            onClick={() => router.visit("/login")}
                        >
                            Login
                        </Button>
                    )}
                </Toolbar>
            </AppBar>

            <Container sx={{ py: 4 }}>{children}</Container>
        </>
    );
}
```

`MainLayout` digunakan oleh semua page agar navigasi dan struktur tampilan tetap konsisten.

3.9 Membuat Halaman Statis React

Buat file berikut:

```
resources/js/pages/About.tsx
resources/js/pages/Guide.tsx
```

Contoh `About.tsx`:

```
import { Paper, Typography } from '@mui/material';
import MainLayout from '../layouts/MainLayout';

export default function About() {
    return (
        <MainLayout>
            <Paper sx={{ p: 4 }}>
                <Typography variant="h4" gutterBottom>
                    About Prezence
                </Typography>
                <Typography>
                    Prezence adalah aplikasi pembelajaran untuk
mengelola presensi siswa.
                </Typography>
            </Paper>
        </MainLayout>
    );
}
```

`Guide.tsx` menggunakan pola yang sama, tetapi memiliki isi yang sesuai untuk petunjuk penggunaan aplikasi.

3.10 Routing Laravel dan Inertia

Buka file:

```
routes/web.php
```

Gunakan `Inertia::render()` untuk menghubungkan route Laravel dengan React page:

```
use Illuminate\Support\Facades\Route;
use Inertia\Inertia;

Route::get('/about', function () {
    return Inertia::render('About');
})->name('about');

Route::get('/guide', function () {
    return Inertia::render('Guide');
})->name('guide');
```

Hubungan route dengan React page:

```
/about
    ↓
Inertia::render('About')
    ↓
resources/js/pages/About.tsx
```

Untuk melihat route yang tersedia:

```
php artisan route:list
```

Pada aplikasi React + Inertia, Laravel tetap menjadi tempat utama untuk mendefinisikan route. Kita tidak menggunakan React Router untuk route aplikasi Laravel.

Latihan Mandiri — Membuat Halaman Contact

Sekarang buat halaman `Contact` secara mandiri. Halaman ini harus dapat dibuka melalui browser pada alamat:

```
http://prezence.test/contact
```

Untuk menyelesaikan latihan ini, buat tiga bagian berikut:

1. React page:

   ```
   resources/js/pages/Contact.tsx
   ```

2. Route Laravel yang mengembalikan page `Contact`.

3. Button `Contact` pada `MainLayout.tsx` yang menggunakan `router.visit('/contact')`.

Gunakan halaman `About` atau `Guide` sebagai referensi. Pastikan Anda memahami hubungan berikut:

```
Button Contact
        ↓
router.visit('/contact')
        ↓
Route Laravel
        ↓
Inertia::render('Contact')
        ↓
Contact.tsx
```

3.11 Migration Dasar Authentication

Laravel menyediakan migration dasar untuk authentication, yaitu tabel:

- `users`;
- `password_reset_tokens`;
- `sessions`.

Laravel Installer biasanya sudah menjalankan migration awal saat project dibuat. Oleh karena itu, periksa status migration terlebih dahulu:

```
php artisan migrate:status
```

Perhatikan kolom **Ran** dan **Migration**. Jika terdapat migration dengan status `Pending`, jalankan:

```
php artisan migrate
```

Jika semua migration sudah berstatus `Ran`, tidak perlu menjalankan `php artisan migrate` kembali.

Pastikan konfigurasi database pada `.env` sesuai dengan database yang digunakan sebelum menjalankan migration.

Pada bab ini kita belum membuat migration untuk `Student`, `Teacher`, `Subject`, `Class`, `TeachingAssignment`, atau `Attendance`.

3.12 Membuat User Awal dengan Seeder

Seeder digunakan untuk memasukkan data awal ke database. Buat seeder khusus untuk akun `admin` menggunakan perintah:

```
php artisan make:seeder AdminSeeder
```

Perintah tersebut membuat file:

```
database/seeders/AdminSeeder.php
```

Buka file `AdminSeeder.php`, lalu isi dengan:

```
namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class AdminSeeder extends Seeder
{
    public function run(): void
    {
        User::updateOrCreate(
            ['email' => 'admin@prezence.test'],
            [
                'name' => 'Administrator',
                'password' => Hash::make('password'),
            ]
        );
    }
}
```

Buka file:

```
database/seeders/DatabaseSeeder.php
```

Panggil `AdminSeeder` dari method `run()`:

```
namespace Database\Seeders;

use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call(AdminSeeder::class);
```

```
    }
}
```

Jalankan seluruh seeder melalui `DatabaseSeeder`:

```
php artisan db:seed
```

Seeder tertentu juga dapat dijalankan secara langsung:

```
php artisan db:seed --class=AdminSeeder
```

Akun pengujian:

```
Email    : admin@prezence.test
Password : password
```

> Password tersebut hanya digunakan untuk pembelajaran lokal. Jangan menggunakan password sederhana seperti ini pada aplikasi produksi.

3.13 Membuat AuthController

Buat controller:

```
php artisan make:controller AuthController
```

Buka file:

```
app/Http/Controllers/AuthController.php
```

Tambahkan method untuk menampilkan form login dan memproses login:

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Inertia\Inertia;

class AuthController extends Controller
{
    public function showLogin()
```

```
    {
        return Inertia::render('Auth/Login');
    }

    public function login(Request $request)
    {
        $credentials = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required', 'string'],
        ]);

        if (Auth::attempt($credentials)) {
            $request->session()->regenerate();

            return redirect()->intended(route('dashboard'));
        }

        return back()->withErrors([
            'email' => 'Email atau password tidak sesuai.',
        ])->onlyInput('email');
    }

    public function logout(Request $request)
    {
        Auth::logout();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect()->route('login');
    }
}
```

Pada bab ini, login menggunakan email dan password sebagai fondasi authentication. Login dengan `mobile_phone` untuk `teacher` dan kombinasi `nis` serta `birth_date` untuk `student` akan dibahas setelah model domain dibuat.

3.14 Membuat Form Login dengan MUI

Buat folder dan file berikut:

```
resources/js/pages/Auth/Login.tsx
```

Isi dengan:

```
import { useForm } from '@inertiajs/react';
import { Alert, Button, Paper, Stack, TextField, Typography } from
'@mui/material';
```

```
import MainLayout from '../../layouts/MainLayout';
import type { FormEvent } from 'react';

export default function Login() {
    const { data, setData, post, processing, errors } = useForm({
        email: '',
        password: '',
    });

    function submit(event: FormEvent<HTMLFormElement>) {
        event.preventDefault();
        post('/login');
    }

    return (
        <MainLayout>
            <Paper sx={{ maxWidth: 480, mx: 'auto', p: 4 }}>
                <Stack component="form" spacing={2} onSubmit={submit}
noValidate>
                    <Typography variant="h4">Login</Typography>

                    {errors.email && <Alert severity="error">
{errors.email}</Alert>}

                    <TextField
                        label="Email"
                        type="email"
                        value={data.email}
                        onChange={(event) => setData('email',
event.target.value)}
                        error={Boolean(errors.email)}
                        helperText={errors.email}
                        required
                    />

                    <TextField
                        label="Password"
                        type="password"
                        value={data.password}
                        onChange={(event) => setData('password',
event.target.value)}
                        error={Boolean(errors.password)}
                        helperText={errors.password}
                        required
                    />

                    <Button type="submit" variant="contained"
disabled={processing}>
                        {processing ? 'Memproses...' : 'Login'}
                    </Button>
                </Stack>
            </Paper>
        </MainLayout>
```

```
    );
}
```

`useForm` dari Inertia membantu mengirim form ke Laravel dan menerima kembali error validasi dari server.

3.15 Membuat DashboardController

Buat controller:

```
php artisan make:controller DashboardController
```

Buka file:

```
app/Http/Controllers/DashboardController.php
```

Isi dengan:

```
namespace App\Http\Controllers;

use Inertia\Inertia;

class DashboardController extends Controller
{
    public function index()
    {
        return Inertia::render('Dashboard');
    }
}
```

Buat file:

```
resources/js/pages/Dashboard.tsx
```

Isi dengan dashboard sederhana:

```
import { Head } from "@inertiajs/react";
import { Paper, Stack, Typography } from "@mui/material";
import MainLayout from "../layouts/MainLayout";

export default function Dashboard() {
    return (
```

```
        <>
            <Head title="Dashboard" />

            <MainLayout>
                <Stack spacing={3}>
                    <Paper sx={{ p: 4 }}>
                        <Typography
variant="h4">Dashboard</Typography>
                        <Typography>
                            Selamat datang di aplikasi Prezence.
                        </Typography>
                    </Paper>
                </Stack>
            </MainLayout>
        </>
    );
}
```

Pada tahap berikutnya, data `auth` akan dibagikan dari Laravel melalui Inertia. Untuk sementara, dashboard cukup menampilkan halaman sederhana.

3.16 Mengenal Middleware `auth`

Middleware adalah lapisan pemeriksaan yang dijalankan sebelum request diteruskan ke controller.

Middleware `auth` memeriksa apakah User sudah login.

```
Browser mengakses /
        ↓
Middleware auth memeriksa session
        ↓
Belum login → redirect ke /login
Sudah login → request diteruskan
        ↓
DashboardController
        ↓
Dashboard React ditampilkan
```

Tambahkan route berikut pada `routes/web.php`:

```
use App\Http\Controllers\AuthController;
use App\Http\Controllers\DashboardController;
use Illuminate\Support\Facades\Route;
use Inertia\Inertia;


Route::get('/about', fn () => Inertia::render('About'))
```

```
    ->name('about');

Route::get('/guide', fn () => Inertia::render('Guide'))
    ->name('guide');

Route::get('/login', [AuthController::class, 'showLogin'])
    ->middleware('guest')
    ->name('login');

Route::post('/login', [AuthController::class, 'login'])
    ->middleware('guest');

Route::middleware('auth')->group(function () {
    Route::get('/', [DashboardController::class, 'index'])
        ->name('dashboard');

    Route::post('/logout', [AuthController::class, 'logout'])
        ->name('logout');
});
```

Route `/` adalah entry point dashboard aplikasi. Jika User belum login, middleware `auth` mengarahkan User ke `/login`. Setelah login berhasil, User diarahkan kembali ke `/`.

3.17 Menggunakan Day.js

Day.js digunakan untuk mengolah data tanggal pada frontend React.

Contoh menampilkan tanggal hari ini:

```
import dayjs from 'dayjs';

const today = dayjs().format('YYYY-MM-DD');
```

Contoh penggunaan pada React state:

```
import { useState } from 'react';
import dayjs from 'dayjs';

const [date, setDate] = useState(dayjs());
```

Pada bab Attendance, Day.js akan digunakan untuk memberikan nilai awal tanggal presensi.

3.18 Menjalankan dan Menguji Aplikasi

Jalankan Vite:

```
npm run dev
```

Kemudian buka:

```
http://prezence.test/about
http://prezence.test/guide
```

Setelah menyelesaikan latihan mandiri, uji halaman Contact:

```
http://prezence.test/contact
```

Menguji Login

1. Buka `http://prezence.test/login`.
2. Masukkan:

   ```
   Email    : admin@prezence.test
   Password : password
   ```

3. Klik **Login**.
4. Pastikan browser diarahkan ke `/`.
5. Pastikan dashboard tampil.

Menguji Middleware

1. Logout dari aplikasi.
2. Buka `http://prezence.test/` secara langsung.
3. Pastikan browser diarahkan ke `/login`.
4. Login kembali.
5. Pastikan browser diarahkan ke `/`.

Menguji Validasi

Coba beberapa kondisi berikut:

- email dikosongkan;
- password dikosongkan;
- email menggunakan format yang salah;
- password salah;
- email belum terdaftar.

Perhatikan pesan validasi yang dikirim kembali oleh Laravel dan ditampilkan oleh React.

Ringkasan Bab Ini

Pada bab ini kita telah:

- membuat project Laravel `Prezence`;
- memasang React dan Inertia;
- memasang MUI v9 dan Day.js;
- mengatur Vite untuk React;
- membuat `MainLayout` dengan MUI;
- membuat page `About` dan `Guide`;
- membuat halaman `Contact` sebagai latihan mandiri;
- memahami hubungan Laravel route dengan Inertia page;
- menjalankan migration dasar authentication;
- membuat User melalui database seeder;
- membuat form login React;
- memvalidasi email dan password melalui Laravel;
- membuat `DashboardController`;
- menggunakan middleware `auth` untuk melindungi route `/`.

Pada bab berikutnya, kita dapat mulai membuat migration dan model domain aplikasi, seperti `Teacher`, `Student`, `Subject`, `Class`, `TeachingAssignment`, dan `Attendance`.